

QUESTION 1: HOSPITAL SHIFT ASSIGNMENT

Question: A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that: (i) D1 cannot work Night, (ii) D2 must work before D3 (Morning < Afternoon < Night), (iii) only one doctor per shift, (iv) D3 cannot work Morning, and (v) all doctors must be assigned exactly one shift. Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule.

Answer and Concept

Backtracking Search builds a solution one decision at a time and checks constraints after each assignment. If a choice violates a constraint, it is undone and another choice is tried. Here, doctors are variables, shifts are their domains, and the rules are constraints.

Constraints: C1: D1 cannot work Night. C2: D2 must be assigned an earlier shift than D3. C3: only one doctor per shift. C4: D3 cannot work Morning. C5: each doctor gets exactly one shift.

Intermediate steps

Step	Assignment	Constraint checks	Decision
1	D1 = Morning	C1 satisfied: D1 is not assigned Night.	Continue
2	D2 = Afternoon	C3 satisfied: D1 and D2 have different shifts.	Continue
3	D3 = Night	C2 satisfied because Afternoon is before Night; C3 and C4 are also satisfied.	Valid solution

Final valid schedule

Shift	Doctor
Morning	D1
Afternoon	D2
Night	D3

Conclusion: The schedule D1–Morning, D2–Afternoon, and D3–Night satisfies all five constraints. Backtracking checks each partial assignment and proceeds only when the constraints remain satisfied.

QUESTION 2: ROBOT NAVIGATION USING BFS

Question: A robot operates in a 5×5 grid and must move from Start (S) to Goal (G). Movement allowed: Up, Down, Left, Right. Each move costs 1. The robot cannot pass through obstacles. Use Manhattan Distance and BFS to show step-by-step node expansion and priority queue updates, and determine the optimal path and cost.

Answer and Concept

Breadth-First Search (BFS) explores nodes level by level using a FIFO queue. When every move has the same cost, BFS finds a shortest path. Manhattan Distance for node n at (r,c) and goal (r_G,c_G) is $h(n) = |r - r_G| + |c - c_G|$. It estimates the minimum horizontal and vertical moves while ignoring obstacles. Standard BFS does not order nodes by this heuristic; a priority queue ordered by $g(n)+h(n)$ is used by A* Search.

Assumption: The source question does not specify exact locations of S, G, or obstacles. The grid below is an illustrative example to demonstrate the algorithm.

Illustrative 5×5 grid

S	.	#	.	.
.	.	#	.	#

#	.	.	.	.
.	#	#	.	.
.	.	.	.	G

Legend: S = Start; G = Goal; # = obstacle; . = free cell. Coordinates are (row, column), numbered 1-5.

BFS node expansion and queue updates

Step	Expanded node	New nodes	Queue after expansion
1	(1,1) S	(2,1)	(2,1)
2	(2,1)	(2,2)	(2,2)
3	(2,2)	(3,2)	(3,2)
4	(3,2)	(3,3)	(3,3)
5	(3,3)	(3,4)	(3,4)
6	(3,4)	(2,4), (4,4), (3,5)	(2,4), (4,4), (3,5)
7	(2,4)	(1,4)	(4,4), (3,5), (1,4)
8	(4,4)	(4,5), (5,4)	(3,5), (1,4), (4,5), (5,4)
9	(3,5)	None	(1,4), (4,5), (5,4)
10	(1,4)	(1,5)	(4,5), (5,4), (1,5)
11	(4,5)	(5,5) G	(5,4), (1,5), (5,5)
12	(5,4)	None	(1,5), (5,5)
13	(1,5)	None	(5,5)
14	(5,5) G	Goal reached	Stop

Optimal path: (1,1) → (2,1) → (2,2) → (3,2) → (3,3) → (3,4) → (4,4) → (5,4) → (5,5).

Path cost: 8 moves × 1 = 8. Manhattan distance from S to G is $|1-5| + |1-5| = 8$. The example has a route achieving this lower bound, so the path is optimal.

Conclusion: BFS finds a shortest path for this illustrative grid because all moves have equal cost. The actual path may differ for a different grid. BFS uses a FIFO queue; the request's mention of a priority queue is more consistent with A* if nodes are prioritized by $g(n)+h(n)$.

QUESTION 3: AUTONOMOUS RESCUE ROBOT

Question: An autonomous rescue robot operates in a partially observable disaster-affected building. The robot must reach a survivor at G from S. It can move up, down, left, or right; some cells are blocked; it observes only adjacent cells; each move costs 1 and entering a risky zone adds 2; it must minimize cost and navigate safely; and it must update its knowledge dynamically. (a) Analyze the constraints. (b) Develop a solution using UCS / Online Search Agent. (c) Explain intelligent agents, rationality, and environment type. (d) Justify the approach.

Answer (a): Constraint analysis

Constraint	Effect on decision-making
Four movement directions	The robot considers up, down, left, and right at each cell.
Blocked cells	Known obstacles must be excluded from feasible routes.
Limited sensing	The complete map is not initially known; local observations are required.
Movement cost = 1	Each ordinary movement adds one unit to the path cost.
Risk premium = 2	Entering a risky cell costs $1 + 2 = 3$ units total.
Safety and minimum cost	The robot compares feasible routes and avoids unsafe or prohibited moves.

Constraint	Effect on decision-making
Dynamic knowledge	New observations update the internal map and may require replanning.

Example: Route A has six ordinary moves and costs 6. Route B has four ordinary moves plus one risky-cell entry and costs $4 + 3 = 7$. If both satisfy the safety requirements, Route A is cheaper.

Answer (b): Solution using UCS and an Online Search Agent

Uniform Cost Search (UCS) expands the node with the smallest accumulated path cost $g(n)$. It uses a priority queue ordered by $g(n)$ and supports non-negative, non-uniform movement costs. An Online Search Agent senses nearby cells, updates its map, acts, and replans as new information becomes available.

Step	Procedure
1. Initialize	Set the start cost to 0 and add S to the priority queue. Begin with the map currently known.
2. Sense	Observe adjacent cells and identify known obstacles, ordinary cells, and risky cells.
3. Generate moves	Consider all four directions and exclude moves outside the grid or into known blocked cells.
4. Calculate costs	Add 1 for an ordinary cell and 3 for a risky cell. Calculate cumulative costs from S.
5. Prioritize	Expand the frontier node with the lowest accumulated cost; update a node if a cheaper path is found.
6. Update and replan	After moving, sense newly visible cells and update the map. Replan if new obstacles or risks change the available routes.
7. Stop	Continue until G is reached by a feasible route while respecting safety constraints.

Illustrative priority queue: Suppose A is an ordinary neighbor at cost 1, B is a risky neighbor at cost 3, and C can be reached through A at cumulative cost 2. The queue orders A (1), C (2), then B (3), unless new observations reveal a cheaper route. This is an example of UCS ordering, not a full rescue route because the assessment does not specify a grid or risky-cell locations.

Answer (c): AI concepts

Concept	Application
Intelligent agent	The robot uses sensors to perceive nearby cells and movement actions to navigate toward the survivor safely.
Rationality	It chooses the best action supported by current information, considers costs and hazards, avoids known obstacles, and replans when new facts appear.
Environment type	Partially observable, sequential, discrete, and non-uniform-cost. The physical environment is dynamic only if hazards or obstacles change; the robot's knowledge changes as it senses new cells.

Answer (d): Justification

UCS is suitable because movement costs are not equal: ordinary moves cost 1 and risky-cell entries cost 3. UCS considers total accumulated cost instead of only the number of moves. An online search agent complements UCS because the robot has only local sensing and must discover the map incrementally. It can update its knowledge and replan after finding obstacles or hazards. Standard UCS guarantees a minimum-cost path when the relevant graph and non-negative costs are known; under partial observability, global optimality cannot be guaranteed from incomplete information alone. Safety restrictions should be enforced explicitly, not only represented as a cost.

Conclusion: Combining cost-based planning with online sensing and replanning is appropriate for rescue navigation under partial observability and risk constraints.

END OF ASSIGNMENT